

Foundations

Of Large Language Models



The Transformer

From “[Attention is All You Need](#)”

Sequence-to-Sequence Network

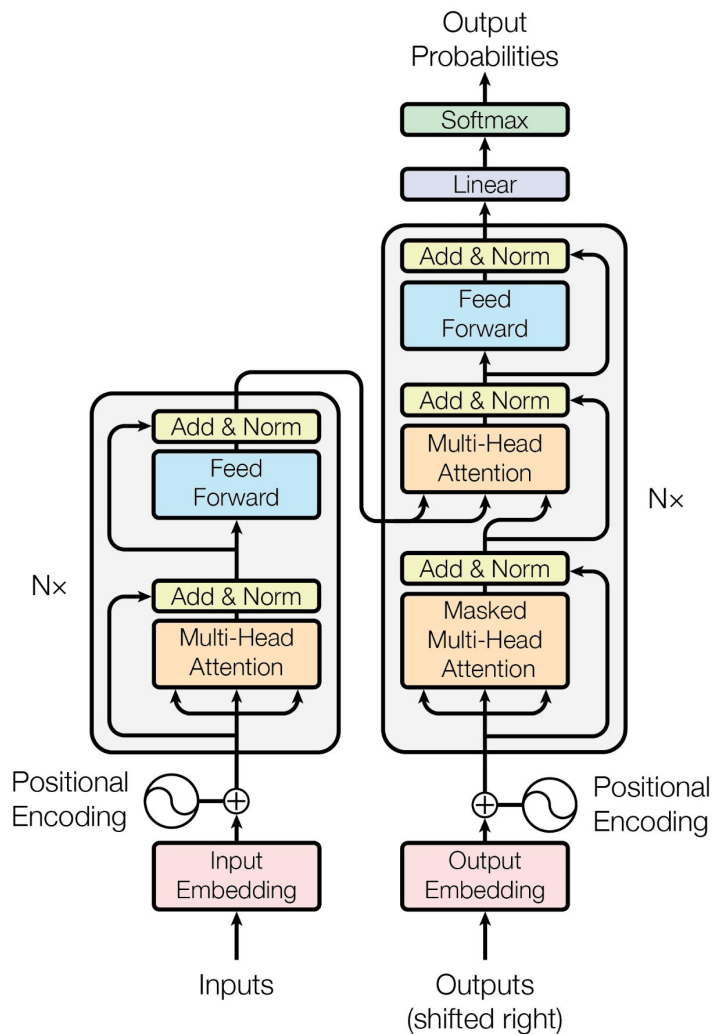
Originally for translation

Autoregressive, but not recurrent

Attention mechanism preserves context awareness

$O(1)$ vs. $O(n)$ in each layer

Solves a multi-class classification problem



Regular self-attention

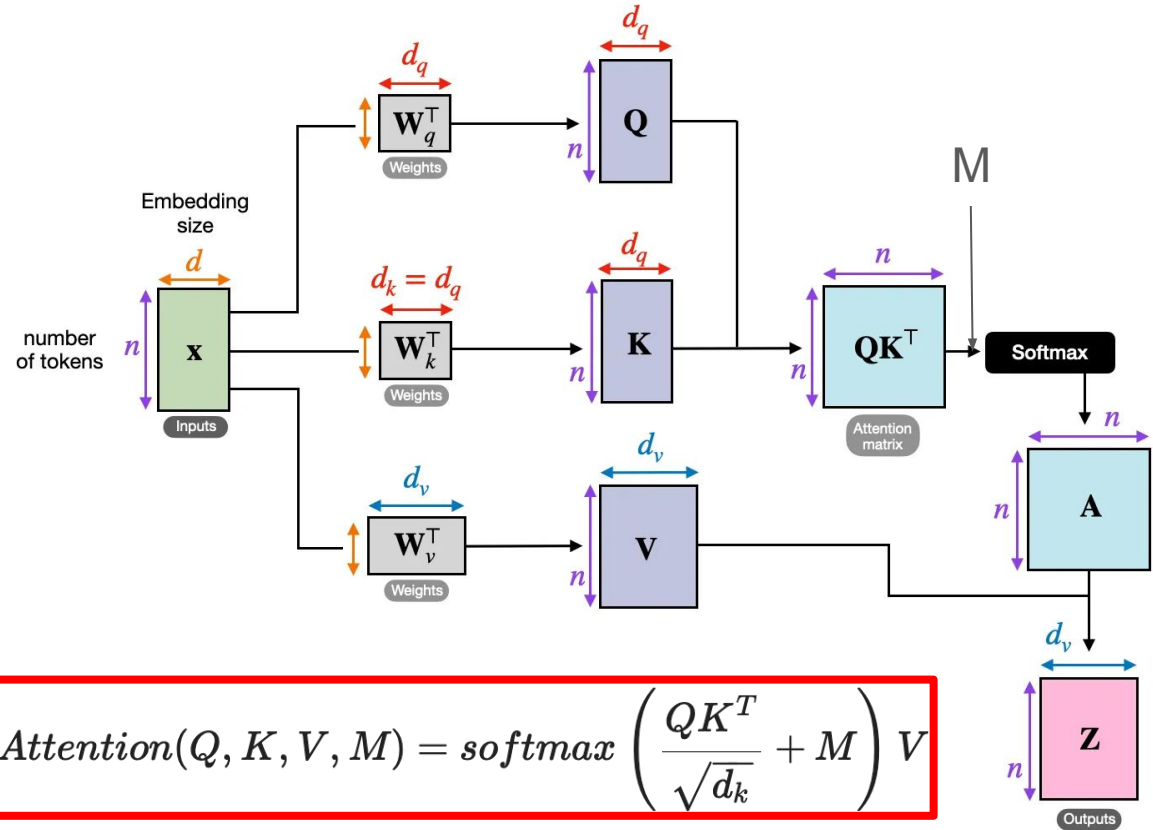
Self - Attention

Compute weight of each token on each other token in sequence

Originally for retrieval and recommendation tasks

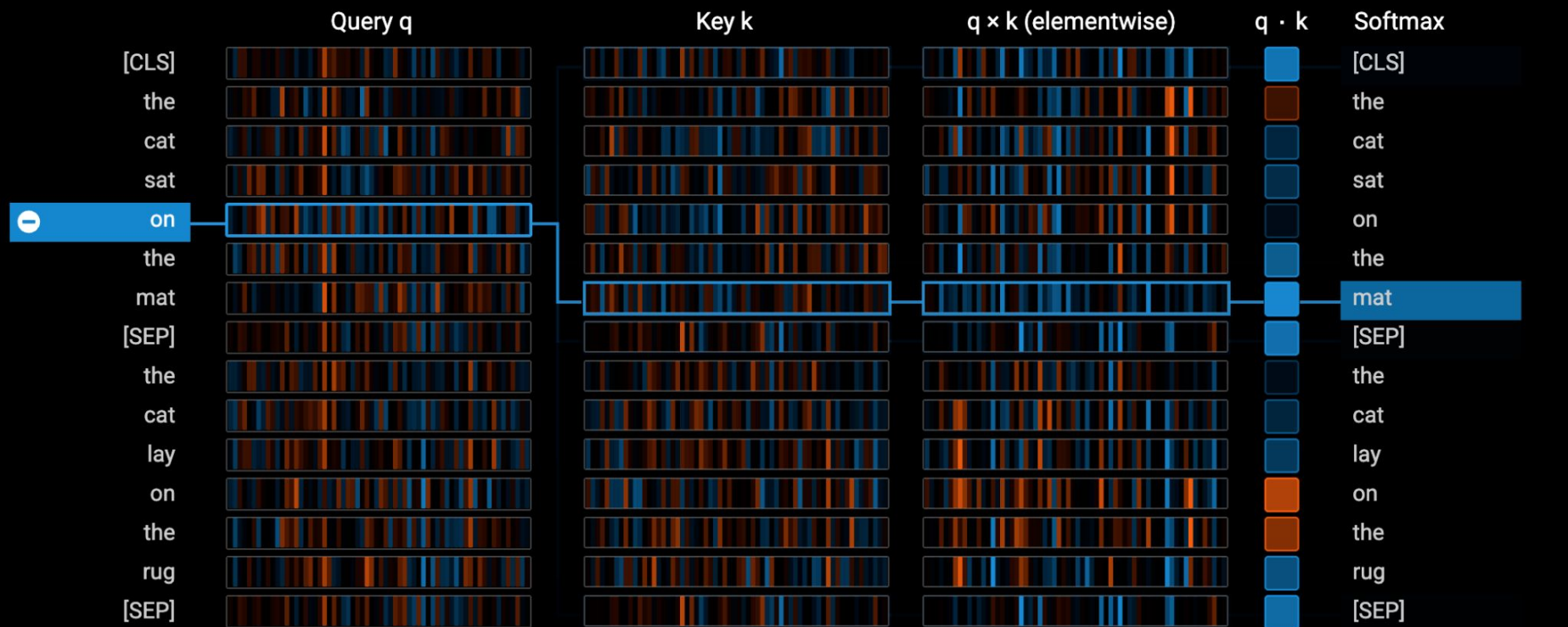
Set of linear operations

Softmax scaling



BertViz

Layer: 4 Head: 5 Attention: All



Cross-Attention

Computes weight of each token across two sequences

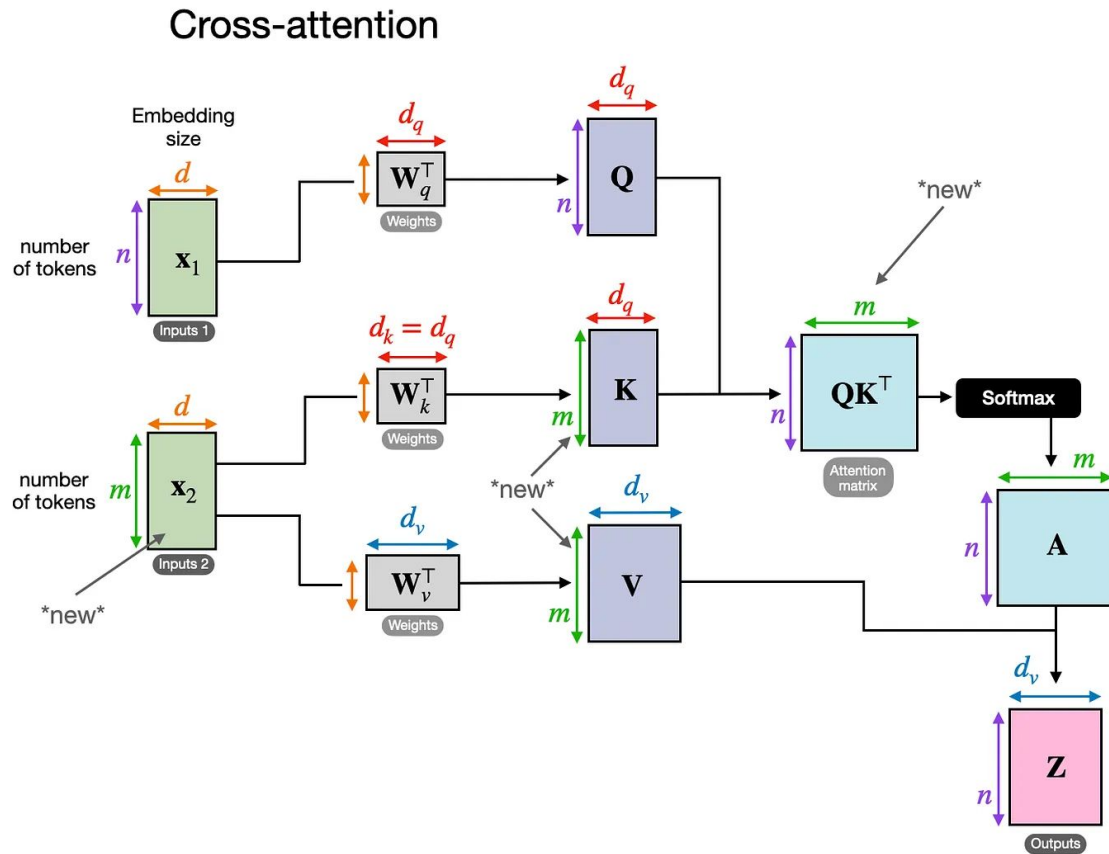
Query comes from decoder

Key, value come from encoder

Used in translation, multimodal applications

Note: No masking in Cross-Attention

Masking happens before in the MHA component.



Llama Innovations - Rotary Position Embedding

$$RoPE(x_m^{(1)}, x_m^{(2)}, m) = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$

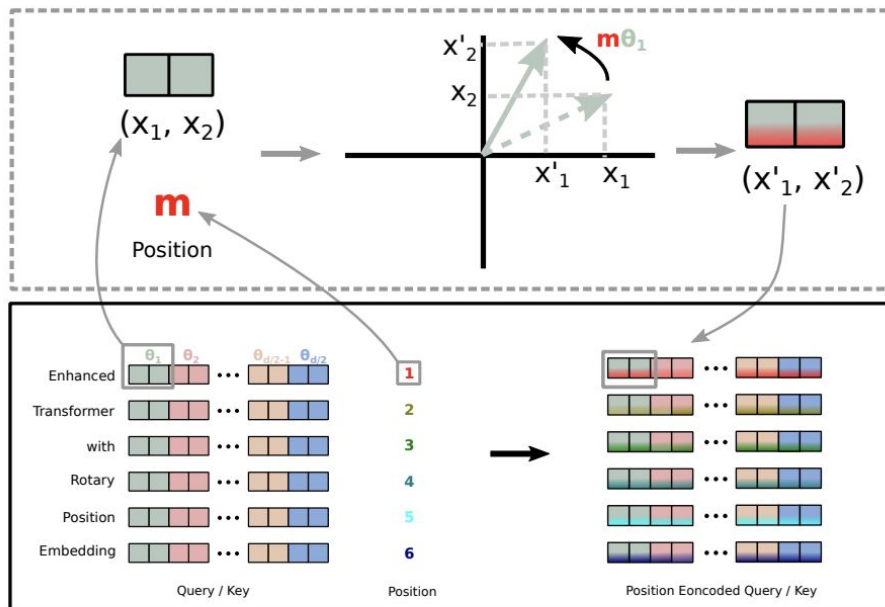
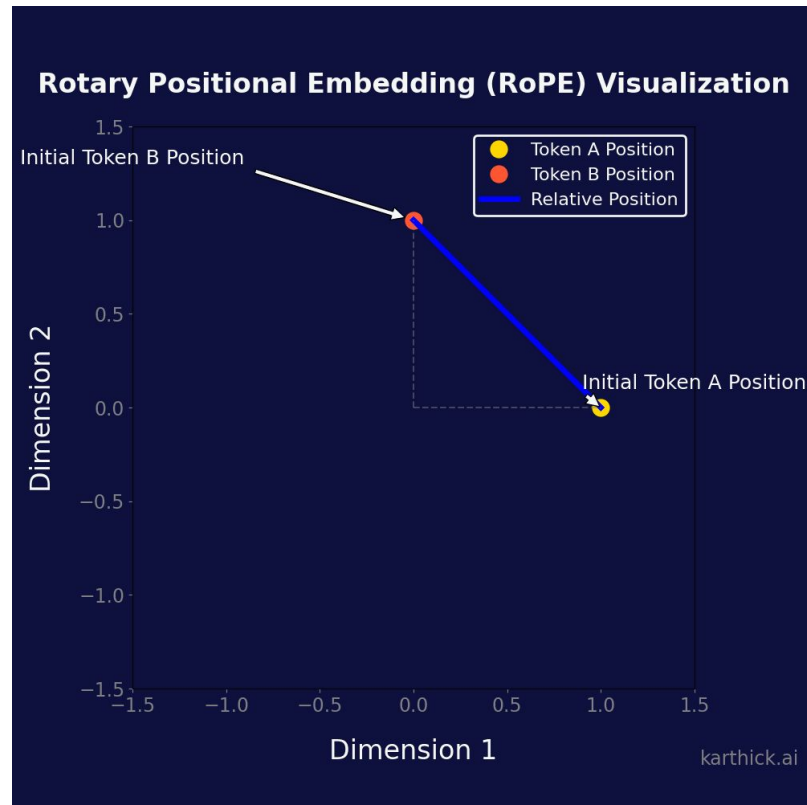
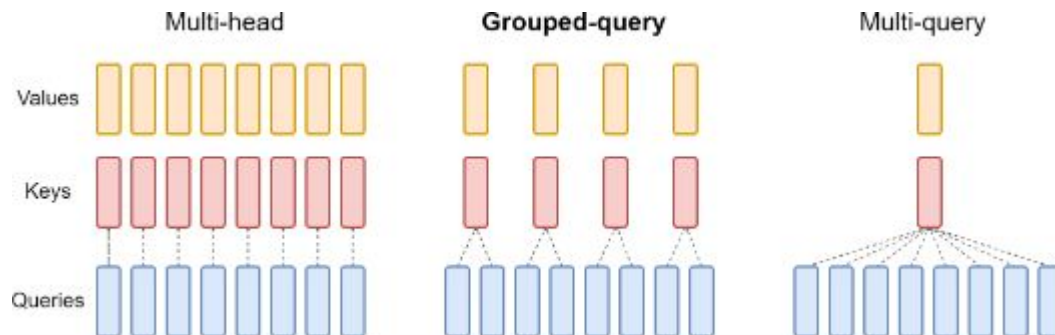


Figure 1: Implementation of Rotary Position Embedding(RoPE).



Llama Innovations

Group Query Attention



RMS Normalization

$$a_i = \frac{a_i}{\sqrt{\frac{1}{n} \sum_i^n a_i^2}} g_i$$

SwiGLU Activation

$$\text{SwiGLU}(A, B) = A \times \text{swish}(B)$$

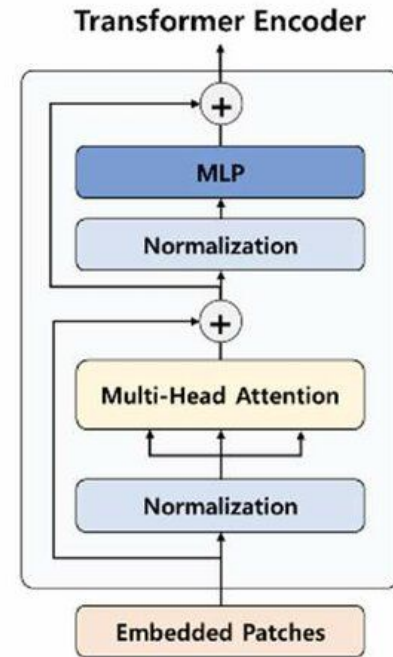
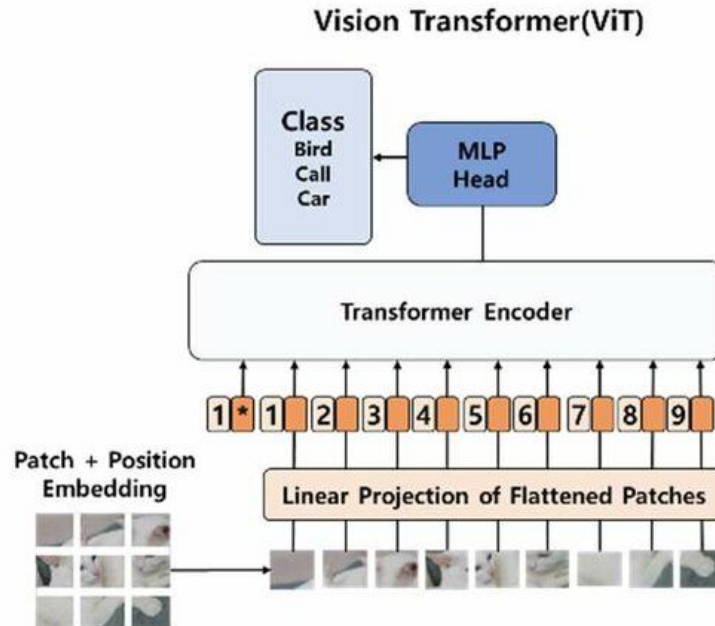
Vision Transformer

From “[An Image is Worth 16x16 words](#): Transformers for Image Recognition at Scale.”

Embeddings represent 16x16 patches in input image

Sequence is Top-Left -> Bottom-Right

Encoder injects representation into cross-attention mechanism or input tokens

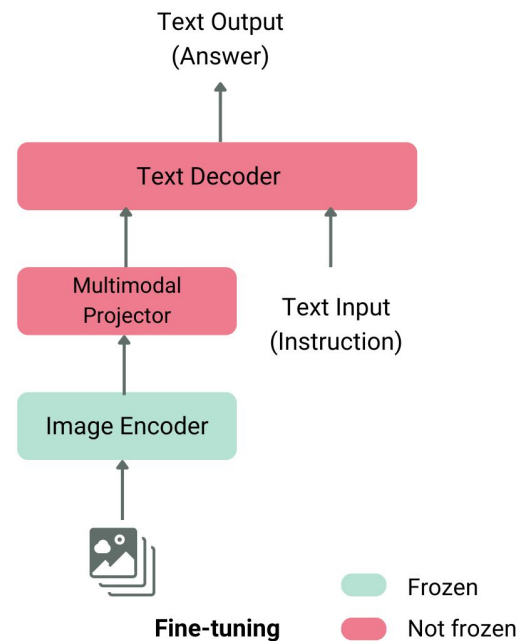
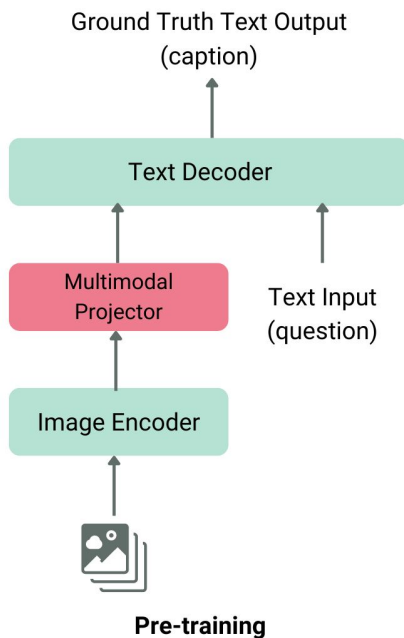


Vision-Language Model

With a unified decoder architecture:

Insert image embeddings as tokens into LLM Prompt

From “[Vision-Language Models Explained](#)”

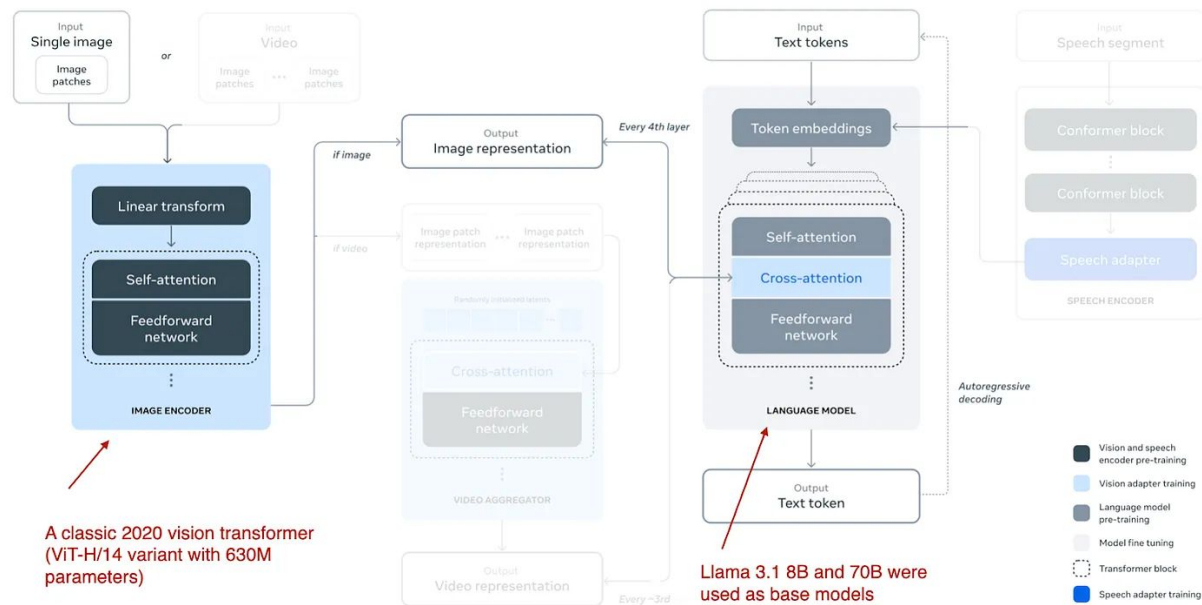


Vision-Language Model

With cross-attention:

Insert image representation as second input for cross-attention in decoder structure.

From [LLama 3 Herd of Models](#)



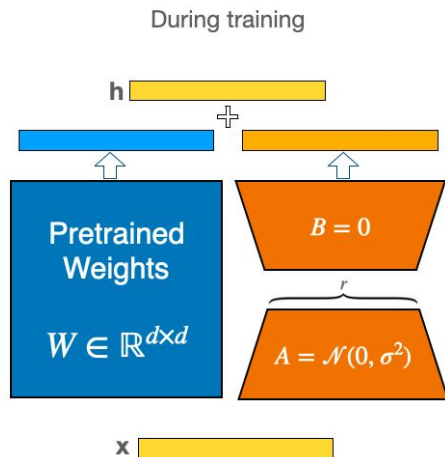
Low-Rank Adaptation

Consider $r \ll d$

Pretrained weights remain fixed

We only tune A and B, hence a significantly smaller number of weights

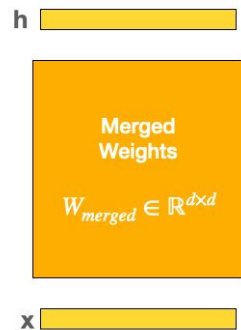
The adapted model contains a sum of the original weights with the newly tuned ones.



$$h = Wx + BAx$$

$$h = \underbrace{(W + BA)}_{W_{merged}}x$$

After training

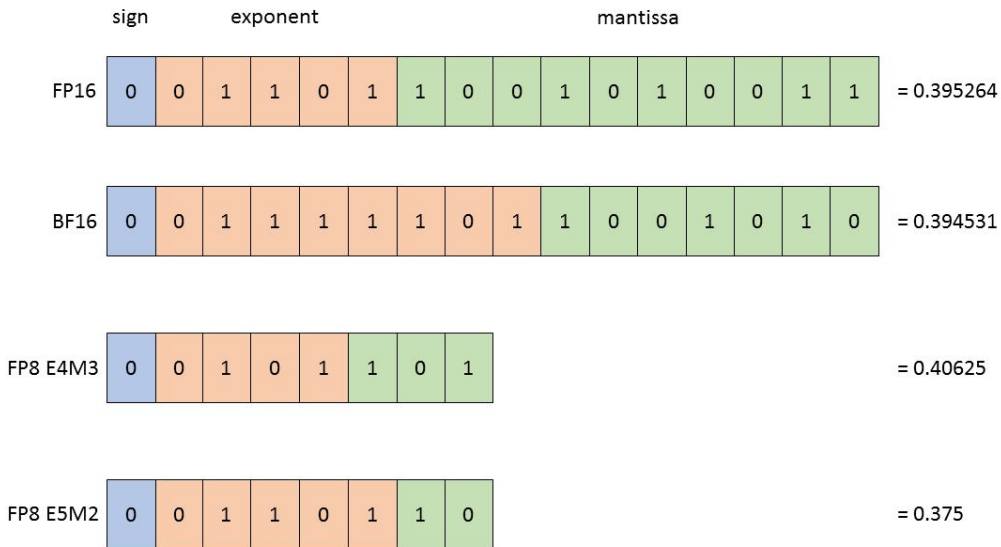


FP representations

The default is FP32, which is computationally intensive.

Modern GPUs and TPUs sacrifice precision (mantissa) for range (exponent) using BF16 format

Latest generation is capable of further optimization with [FP8](#)



Fundamental Equations

1. Linear Projection

$$y = Wx + b$$

2. Weight Update

$$w_{t+1} = w_t - \eta \nabla \mathcal{L}(w, x)$$

3. Chain Rule

$$\frac{df(g(x))}{dx} = \frac{df(g)}{dg} \frac{dg(x)}{d(x)}$$

4. Cross-Entropy

$$H(p, q) = - \sum_x p(x) \log(q(x))$$

Fundamental Equations

1. Softmax

$$\sigma(z)_i = \frac{e^{z_i/kT}}{\sum_{j=1}^N e^{z_j/kT}}$$

2. Swish Activation

$$\textit{swish}(x) = \frac{x}{1 + e^{-\beta x}}$$

3. SwiGLU

$$\textit{SwiGLU}(A, B) = A \times \textit{swish}(B)$$

Fundamental Equations

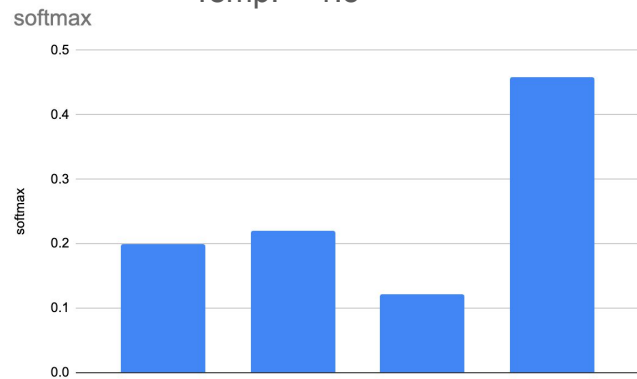
Rotary Position Embeddings

$$RoPE(x_m^{(1)}, x_m^{(2)}, m) = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix} \begin{pmatrix} x_m^{(1)} \\ x_m^{(2)} \end{pmatrix}$$

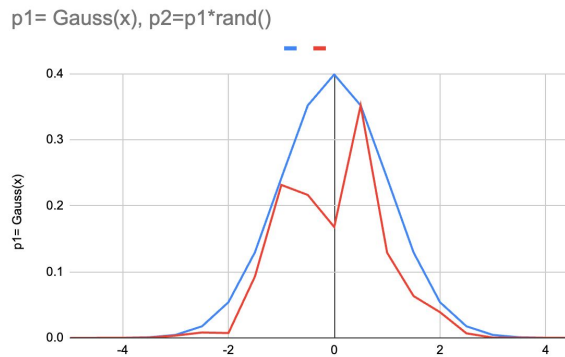
RMS Normalization

$$a_i = \frac{a_i}{\sqrt{\frac{1}{n} \sum_i^n a_i^2}} g_i$$

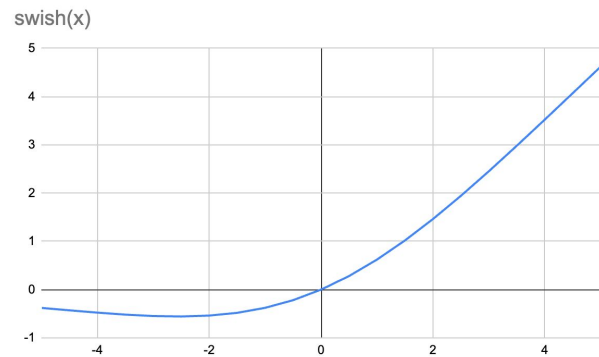
Temp. = 1.5



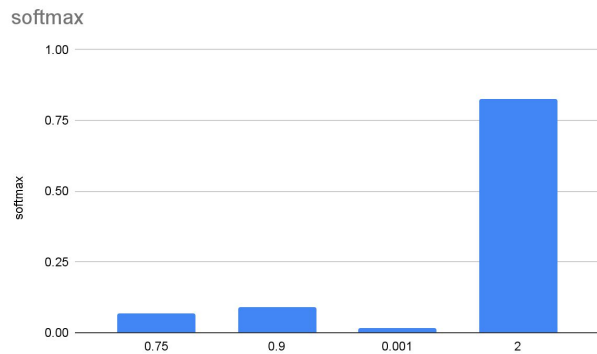
$H(p_1, p_2) = 1.66$



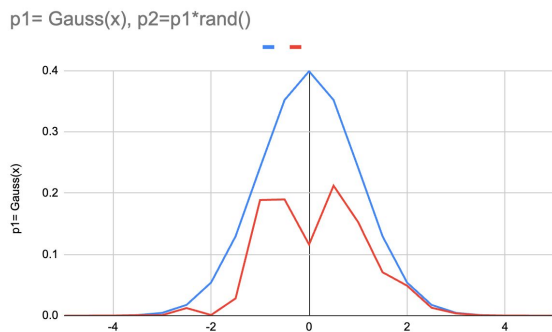
Beta=0.5



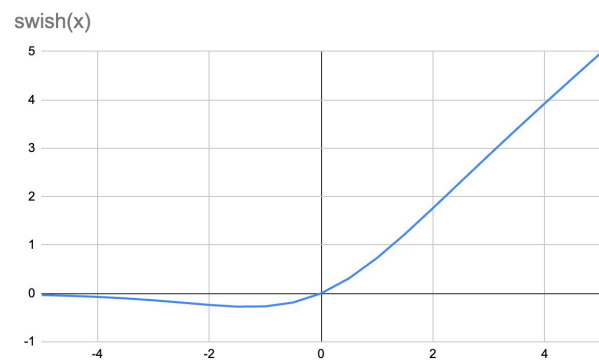
Temp. = 0.5



$H(p_1, p_2) = 1.917$



Beta=1.0



LLM Foundations Labs

1. <https://www.kaggle.com/code/gmarchetti/1-gpt-tf-basic>
2. <https://www.kaggle.com/code/gmarchetti/2-llama2-simplebooks>
3. <https://www.kaggle.com/code/gmarchetti/3-build-vlm>
4. <https://www.kaggle.com/code/gmarchetti/4-finetune-llama32>
5. Placeholder for another notebook
6. Placeholder for another notebook
7. <https://www.kaggle.com/code/pratapram/7-pytorch-llm-train-ipynb>
8. <https://github.com/gmarchetti2020/llm-foundations>